

Chapter 4

Database Connectivity With Java

JDBC

JDBC stands for Java Database connectivity. Its advancement for ODBC (Open Database Connectivity). JDBC is a standard API specification developed in order to move data from frontend to backend. This API consists of classes and interface written in java. It is basically acts as an interface(not the one we use in java) or channel between our java program and databases i.e it establishes a link between the two so that a programmer could send data from java code and store it in the database for future use.

Steps for connectivity between java program and database

1. Loading the Driver

To begin with, we first need load the driver or register it before using it in the program Registration is to be done once in our program. We can register a driver in a way mentioned before.

2. Class.forName()

Here we load the driver's class file into memory at the runtime. No need of using new or creation of object. The following examples uses Class.forName() to load the Mysql driver

```
Class.forName("com.mysql.jdbc.Driver");
```

3. Create the connection

```
Connection con = DriverManager.getConnection(url, user, password)
```

User – username from which our sql command prompt can be accessed.

Password – password from which our sql command prompt can be accessed.

Con – it is a reference to connection interface.

Url – Uniform Resource Locator. It can be created as follows:

```
String url = "jdbc:mysql://localhost:3306/database_name"
```

4. Create a statement

Once a connection is established we can interact with the database. The JDBCStatement, CallableStatement and PreparedStatement interfaces define the methods that enable us to send

SQL commands and receive data from our database.

5. Use of JDBC Statement is as follows

```
Statement st = con.createStatement();
```

Here, con is a referenced to connection interface used in previous step.

6. Execute the Query

The executeQuery() method of statement interface is used to execute queries of retrieving values from the database. This method returns the object of ResultSet that can be used to get all the records of a table. The executeUpdate(sql query) method of Statement interface is used to execute queries of updating / inserting.

Example:

```
Int m = st.executeUpdate(sql);  
If (m == 1)  
    System.out.println("inserted successfully:" +sql);  
Else  
    System.out.println("insertion failed");
```

Here sql is sql query of the type string.

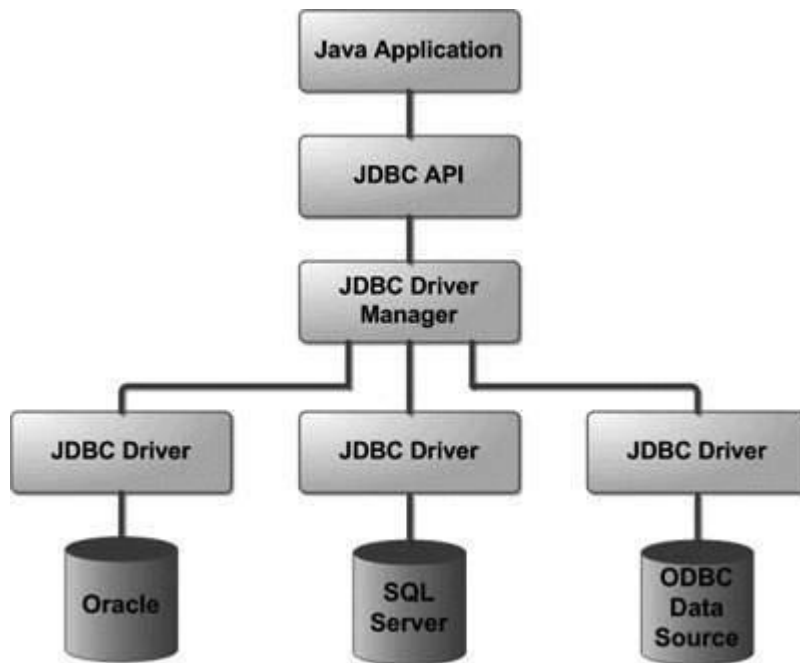
7. Close the connection

By closing connection, objects of Statement and Resultset will be closed automatically. The close() method of Connection interface is used to close the connection.

Example:

```
Con.close();
```

JDBC Architecture



The JDBC API supports both two tier and three tier processing models for database access but in general, JDBC Architecture consists of two layers:

- **JDBC API:** This provides the application – to – JDBC Manager connection.
- **JDBC Driver API:** This supports the JDBC Manager – to – Driver Connection.

The JDBC API uses a driver manager and database – specific drivers to provide transparent connectivity to heterogeneous databases. The JDBC driver manager ensures that the correct driver is used to access each data source. The driver manager is capable of supporting multiple concurrent drivers connected to multiple heterogeneous databases.

Common JDBC Components

- **DriverManager** – This class manages a list of database drivers. Matches connection requests from the java application with the proper database driver using communication sub protocol. The first driver that recognizes a certain subprotocol under JDBC will be used to establish a database Connection.
- **Driver** – This interface handles the communications with the database server. You will interact directly with Driver objects very rarely. Instead, you use DriverManager objects, which manages objects of this type. It also abstracts the details associated with working with Driver objects.
- **Connection** – This interface with all methods for contacting a database. The connection object represents communication context, i.e., all communication with database is through connection object only.
- **Statement** – You use objects created from this interface to submit the SQL statements to the database. Some derived interfaces accept parameters in addition to executing stored procedures.
- **ResultSet** – These objects hold data retrieved from a database after you execute an SQL query using Statement objects. It acts as an iterator to allow you to move through its data.
- **SQLException** – This class handles any errors that occur in a database application.

Example 1 (Get data from table)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

class Select1
{
    public static void main(String args[])
    {
        getData();
    }

    public static Connection getConnection() {
        try {
            String driver = "com.mysql.cj.jdbc.Driver";
            String databaseUrl = "jdbc:mysql://localhost:3306/Nesfield";
            String username = "root";
            String password = "";
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(databaseUrl, username, password);
            System.out.println("Database connected");
            return conn;
        }
        catch (Exception e)
        {
            System.out.println("Some error: " + e);
        }
        return null;
    }

    public static void getData()
    {
        try
        {
            Statement statement = getConnection().createStatement();
            ResultSet result = statement.executeQuery("SELECT * FROM nesfield_student");
            while (result.next()) {
                System.out.println(result.getString("Roll_no"));
                System.out.println(result.getString("Name"));
                System.out.println(result.getString("Email"));
                System.out.println(result.getString("Address"));
            }
        }
        catch (Exception e)
        {
            System.out.println("Error: " + e);
        }
    }
}
```

Output

1

Nishanta

Nishanta@gmail.com

Dang

2

Hari

Hari@gmail.com

Kathmandu

3

Shyam

Shyam@gmail.com

Lalitpur

Example: 2 (Insert data into Table)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class Insertion
{
    public static void main(String args[])
    {
        InsertData();
    }

    public static Connection getConnection()
    {
        try
        {
            String driver = "com.mysql.cj.jdbc.Driver";
            String databaseUrl = "jdbc:mysql://localhost:3306/Nesfield";
            String username = "root";
            String password = "";
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(databaseUrl, username, password);
            System.out.println("Database connected");
            return conn;
        }
        catch (Exception e) {
            System.out.println("Some error: " + e);
        }
        return null;
    }

    public static void InsertData()
    {
        try
        {
            Statement statement = getConnection().createStatement();
            String query = "INSERT INTO nesfield_student (Roll_no, Name, Email, Address) VALUES " +
                "(4, 'Raju', 'Raju@gmail.com', 'Baglung'), " +
                "(5, 'Harry', 'Harry@gmail.com', 'Ghusel)";
            int result = statement.executeUpdate(query);
            System.out.println(result + " row(s) inserted.");
        }
        catch (Exception e)
        {
            System.out.println("Error: " + e);
        }
    }
}
```

Output:

Roll_no	Name	Email	Address
1	Nishanta	Nishanta@gmail.com	Dang
2	Hari	Hari@gmail.com	Kathmandu
3	Shyam	Shyam@gmail.com	Lalitpur
4	Raju	Raju@gmail.com	Baglung
5	Harry	Harry@gmail.com	Ghusel

Example: 3 (Update data into table)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class Update
{
    public static void main(String args[])
    {
        updateData();
    }

    public static Connection getConnection()
    {
        try
        {
            String driver = "com.mysql.cj.jdbc.Driver";
            String databaseUrl = "jdbc:mysql://localhost:3306/Nesfield";
            String username = "root";
            String password = "";
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(databaseUrl, username, password);
            System.out.println("Database connected");
            return conn;
        }
        catch (Exception e) {
            System.out.println("Some error: " + e);
        }
        return null;
    }
}
```

```

public static void updateData()
{
    try
    {
        Statement statement = getConnection().createStatement();
        String query = "UPDATE nesfield_student SET name = 'Zenith' WHERE Roll_no = 3";
        int result = statement.executeUpdate(query);
        System.out.println(result + " row(s) updated.");
    }
    catch(Exception e)
    {
        System.out.println("Error: " + e);
    }
}
}

```

Output:

Roll_no	Name	Email	Address
1	Nishanta	Nishanta@gmail.com	Dang
2	Hari	Hari@gmail.com	Kathmandu
3	Zenith	Shyam@gmail.com	Lalitpur
4	Raju	Raju@gmail.com	Baglung
5	Harry	Harry@gmail.com	Ghusel

Example: 4 (Delete data from database)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class Deletion
{
    public static void main(String args[])
    {
        deleteData();
    }

    public static Connection getConnection()
    {
        try
        {
            String driver = "com.mysql.cj.jdbc.Driver";
            String databaseUrl = "jdbc:mysql://localhost:3306/Nesfield";
            String username = "root";
            String password = "";
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(databaseUrl, username, password);
            System.out.println("Database connected");
            return conn;
        }
        catch (Exception e) {
            System.out.println("Some error: " + e);
        }
        return null;
    }

    public static void deleteData()
    {
        try
        {
            Statement statement = getConnection().createStatement();
            String query = "Delete from nesfield_student where roll_no = 4";
            int result = statement.executeUpdate(query);
            System.out.println(result + " row(s) Deleted.");
        }
        catch (Exception e)
        {
            System.out.println("Error: " + e);
        }
    }
}
```

Output

Roll_no	Name	Email	Address
1	Nishanta	Nishanta@gmail.com	Dang
2	Hari	Hari@gmail.com	Kathmandu
3	Zenith	Shyam@gmail.com	Lalitpur
5	Harry	Harry@gmail.com	Ghusel

Example: 5 (Get record with input from mysql)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
import java.util.Scanner;

public class GetFromUser
{
    public static void main(String args[])
    {
        Get_Data_User();
    }
    public static Connection getConnection()
    {
        try{
            String driver = "com.mysql.cj.jdbc.Driver";
            String databaseUrl = "jdbc:mysql://localhost:3306/Nesfield";
            String username = "root";
            String password = "";
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(databaseUrl, username, password);
            System.out.println("Database connected");
            return conn;
        }
        catch (Exception e)
        {
            System.out.println("Some error: " + e);
        }
        return null;
    }
}
```

```

public static void Get_Data_User()
{
    try
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter your roll number");
        int id = sc.nextInt();

        Statement statement = getConnection().createStatement();
        ResultSet result = statement.executeQuery("SELECT * FROM nesfield_student where roll_no = " + id);
        while (result.next())
        {
            System.out.println(result.getString("Roll_no"));
            System.out.println(result.getString("Name"));
            System.out.println(result.getString("Email"));
            System.out.println(result.getString("Address"));
        }
    }
    catch (Exception e)
    {
        System.out.println("Error: " + e);
    }
}
}

```

Output

Enter your roll number

3

Database connected

3

Zenith

Shyam@gmail.com

Lalitpur

Example: 6 (create table)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class Create_table
{
    public static void main(String args[])
    {
        createTable();
    }

    public static Connection getConnection()
    {
        try
        {
            String driver = "com.mysql.cj.jdbc.Driver";
            String databaseUrl = "jdbc:mysql://localhost:3306/Nesfield";
            String username = "root";
            String password = "";
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(databaseUrl, username, password);
            System.out.println("Database connected");
            return conn;
        }
        catch (Exception e) {
            System.out.println("Some error: " + e);
        }
        return null;
    }
}
```

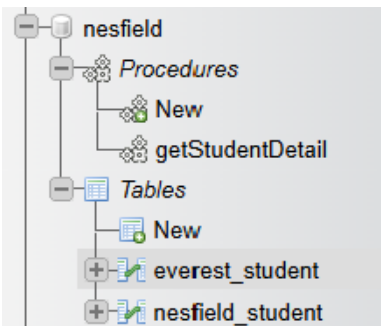
```

public static void createTable()
{
    try
    {
        Statement statement = getConnection().createStatement();
        String query = "CREATE TABLE Everest_Student ("
            + "id INT, "
            + "name VARCHAR(20), "
            + "faculty VARCHAR(20), "
            + "phone VARCHAR(20)"
            + ")";

        int result = statement.executeUpdate(query);
        System.out.println("Table created");
    }
    catch(Exception e)
    {
        System.out.println("Error: " + e);
    }
}
}

```

Output

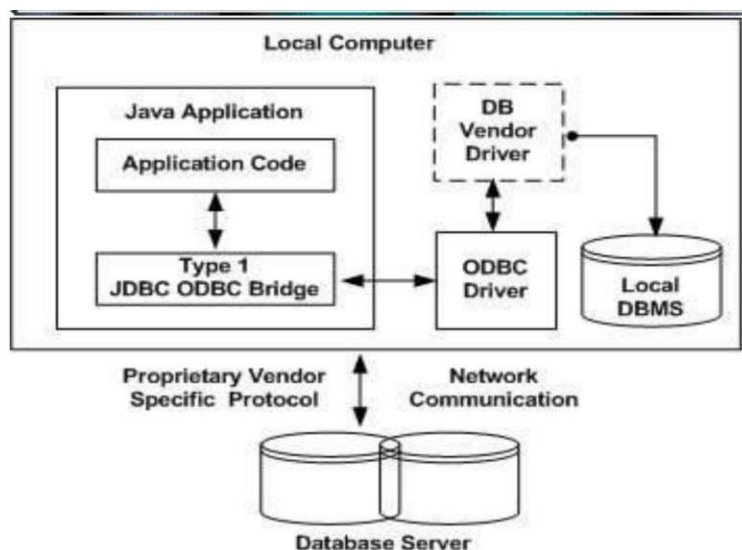


JDBC Driver Types

JDBC driver implementations vary because of the wide variety of operating systems and hardware platforms in which Java operates. Sun has divided the implementation types into four categories, Types 1, 2, 3 and 4 which is explained below.

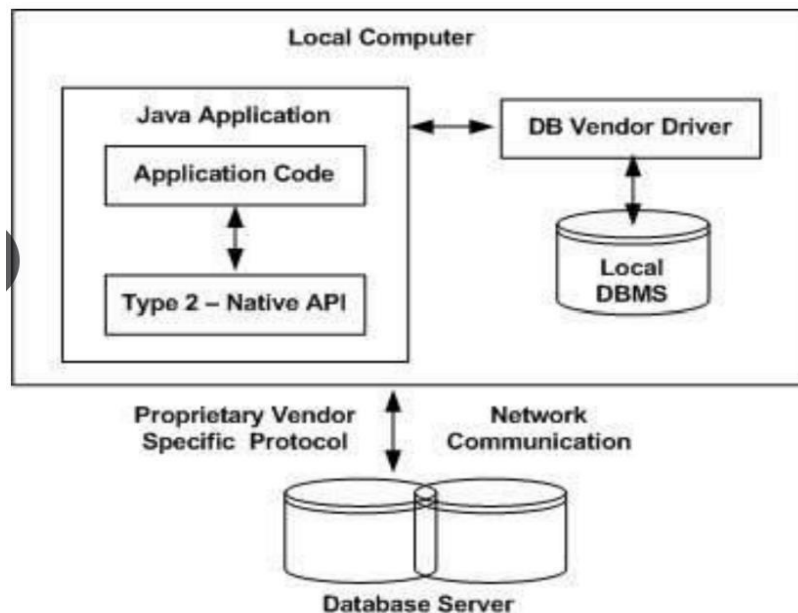
Type 1: JDBC – ODBC Bridge Driver

In a Type 1 driver, a JDBC bridge is used to access ODBC drivers installed on each client machine. Using ODBC, requires configuring on our system a Data Source Name(DSN) that represents the target database. When java first came out, this was a useful driver because most databases only supported ODBC access but now **this type of driver is recommended only for experimental use or when no other alternative is available.**



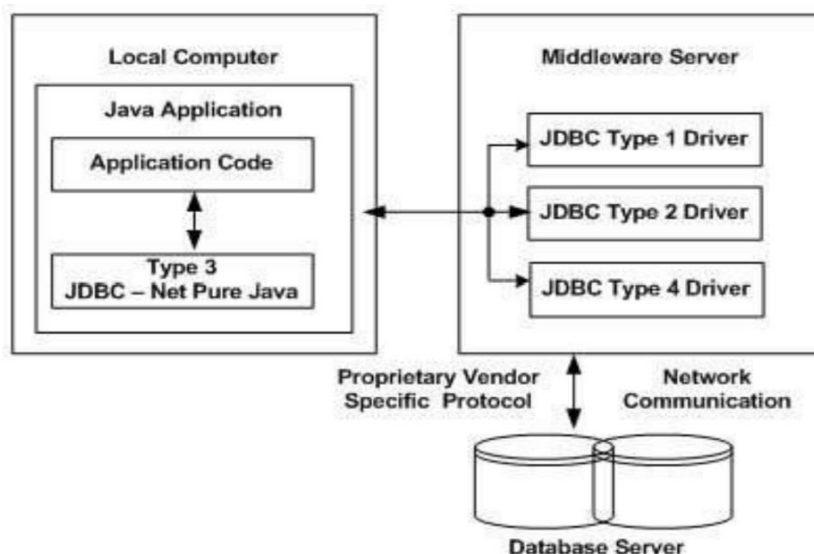
Type 2: JDBC – Native API

In this Type 2 driver, JDBC API calls are converted into native C / C++ API calls, which are unique to the database. These drivers are typically provided by the database vendors and used in the same manner as the JDBC – ODBC bridge. The vendor – specific driver must be installed on each client machine. If we change the Database, we have to change the native API, as it is specific to a database and they are mostly obsolete now, but we may realize some speed increase with a Type 2 driver because it eliminates ODBC's overhead.



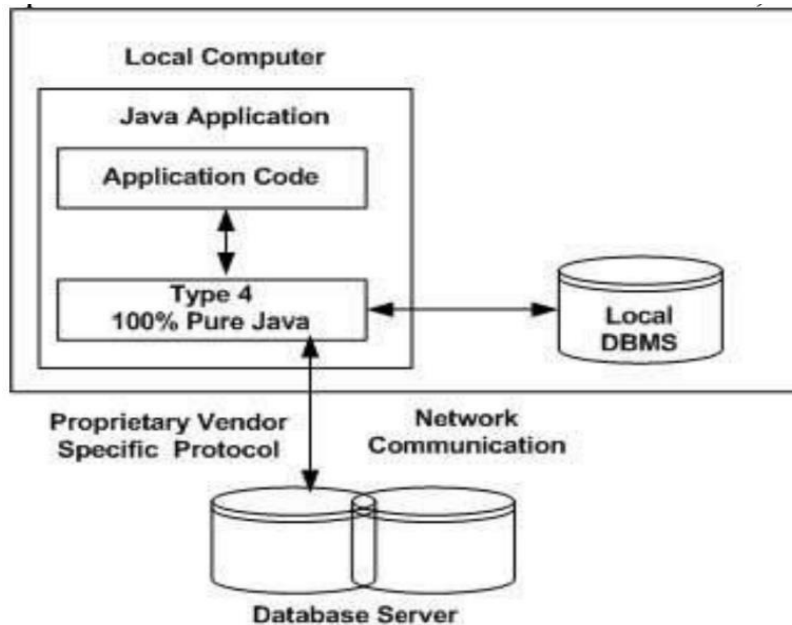
Type 3: JDBC – Net pure java

In a Type 3 driver, a three – tier approach is used to access databases. The JDBC clients use standard network sockets to communicate with a middleware application server. The socket information then translated by the middleware application server into the call format required by the DBMS, and forwarded to the database server. This kind of driver is extremely flexible, since it requires no code installed on the client and a single driver can actually provide access to multiple databases.



Type 4: 100 % pure java

In a Type 4 driver, a pure java – based driver communicates directly with the vendor’s database through socket connection. This is the highest performance driver available for the database and is usually provided by the vendor itself. This kind of driver is extremely flexible; we don’t need to install special software on the client or server. Further these drivers can be downloaded dynamically.



MySQL’s Connector / J driver is a Type 4 driver. Because of the proprietary nature of their network protocols, database vendors usually supply type 4 drivers.

Creating a statement for Executing Queries

Once a connection is obtained we can interact with the database. The JDBC Statement, Callable statement and prepared statement interfaces define the methods and properties that enable us to send SQL or PL/SQL command and receive data from our database. They also define methods that help bridge data type differences between java and SQL data types used in database.

1. Creating Statement object

Before using a statement object to execute a SQL statement, we need to create one using the Connection object's **createStatement()** method.

```
Statement st = null;

try
{
    st = conn.createStatement();

    .....
}
catch(SQLException e)
{
    .....
}

Finally
{
    .....
}
```

2. Prepared Statement object

The PreparedStatement interface extends the statement interface, which gives us added functionality with a couple of advantages over a generic Statement object. This statement gives us the flexibility of supplying arguments dynamically.

```
PreparedStatement ps = null;

try
{
    String SQL = "Update Employee SET age = ? where id = ?";
    ps = conn.prepareStatement(SQL);
}
catch(SQLException e)
{
    .....
}

Finally
{
    .....
}
```

Example:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class prepared_statement1
{
    public static void main(String args[])
    {
        UpdateData();
    }
    public static Connection getConnection()
    {
        try
        {
            String driver = "com.mysql.cj.jdbc.Driver";
            String databaseUrl = "jdbc:mysql://localhost:3306/nesfield";
            String username = "root";
            String password = "";
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(databaseUrl, username, password);
            System.out.println("Database connected");
            return conn;
        }
        catch (Exception e)
        {
            System.out.println("Some error: " + e);
        }
        return null;
    }
}
```

```

public static void UpdateData()
{
    try
    {
        Connection conn = getConnection();
        {
            String sql = "UPDATE nesfield_student SET address = ? WHERE Roll_no = ?";
            PreparedStatement ps = conn.prepareStatement(sql);

            ps.setString(1, "Bhaktapur");
            ps.setInt(2, 1);
            ps.executeUpdate();

            ps.setString(1, "Dhangadi");
            ps.setInt(2, 3);
            ps.executeUpdate();

            ps.close();
            conn.close();
        }
    }
    catch (Exception e)
    {
        System.out.println("Error: " + e);
    }
}

```

Output

Roll_no	Name	Email	Address
1	Nishanta	Nishanta@gmail.com	Bhaktapur
2	Hari	Hari@gmail.com	Kathmandu
3	Zenith	Shyam@gmail.com	Dhangadi
5	Harry	Harry@gmail.com	Ghusel

All the parameters in JDBC are represented by the ? Symbol, which is known as the parameter marker. We must supply values for every parameter before executing

3. Callable Statement object

Just as a Connection object creates the Statement and PreparedStatement objects, it also creates the CallableStatement object, which would be used to execute a call to a database store procedure.

```
CREATE OR REPLACE PROCEDURE getEmpName
(EMP_ID IN NUMBER, EMP_FIRST OUT VARCHAR)
as BEGIN
SELECT first INTO_FIRST
FROM Employees WHERE
ID = EMP_ID
END;
```

Above stored procedure has been written for Oracle, but we are working with MySQL database so, let us write same stored procedure for MySQL as follows to create it in EMP database.

```
DELIMITER $$
DROP PROCEDURE IF EXISTS 'EMP'.'getEmpName'$$
CREATE PROCEDURE 'EMP'.'getEmpName'
(IN EMP_ID INT, OUT EMP_FIRST VARCHAR(255))
BEGIN
SELECT first INTO EMP_FIRST
FROM Employees WHERE
ID = EMP_ID;
END $$
DELIMITER
R
```

Three types of parameter exist: IN, OUT and INOUT. The PreparedStatement object only uses the IN parameter. The CallableStatement object can use all the three.

Example

```
CallableStatement cs = null;

try
{
    String SQL = "{call getEmpName(?,?)}";
    cs = conn.prepareCall(SQL);
}
catch(SQLException e)
{
    .....
}

Finally
{
    .....
}
```

Example:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.CallableStatement;
import java.sql.ResultSet;

public class callable_statement1
{
    public static void main(String args[])
    {
        selectData();
    }

    public static Connection getConnection()
    {
        try {
            String driver = "com.mysql.cj.jdbc.Driver";
            String databaseUrl = "jdbc:mysql://localhost:3306/nesfield";
            String username = "root";
            String password = "";
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(databaseUrl, username, password);
            System.out.println("Database connected");
            return conn;
        }
        catch (Exception e)
        {
            System.out.println("Some error: " + e);
        }
        return null;
    }
}
```

```

public static void selectData()
{
    try
    {
        Connection conn = getConnection(); // Establish the connection
        {
            String sql = "call getStudentDetail()";
            CallableStatement cs = conn.prepareCall(sql);

            ResultSet result = cs.executeQuery();
            while (result.next())
            {
                System.out.println(result.getString("roll_no"));
                System.out.println(result.getString("name"));
                System.out.println(result.getString("email"));
                System.out.println(result.getString("address"));
            }
        }
    }
    catch (Exception e)
    {
        System.out.println("Error: " + e);
    }
}

```

Output

Database connected

1

Nishanta

Nishanta@gmail.com

Bhaktapur

2

Hari

Hari@gmail.com

Kathmandu

3

Zenith

Shyam@gmail.com

Dhangadi

5

Harry

Harry@gmail.com

Ghusel

SQL Exception

- JDBC Exception handling is very similar to the java Exception handling but for JDBC, the most common exception you'll deal with is java.sql.SQLException.
- An SQLException can occur both the driver and the database.
- When such an exception occurs, an object of type SQLException will be passed to the clause.
- By utilizing the information available from the exception object we can catch an exception and continue our program appropriately.

Example

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.SQLException;
```

```
public class Insertion{
    public static void main(String args[])
    {
        InsertData();
    }
    public static Connection getConnection() {
        try {
            String driver = "com.mysql.cj.jdbc.Driver";
            String databaseUrl = "jdbc:mysql://localhost:3306/Nesfield";
            String username = "root";
            String password = "";
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(databaseUrl, username, password);
            System.out.println("Database connected");
            return conn;
        }
        catch (SQLException e) {
            System.out.println("Some error: " + e);
        }
        catch (ClassNotFoundException e){
            System.out.println("wrong class Name");
        }
        return null;
    }
}
```

```

public static void InsertData()
{
    try
    {
        Statement statement = getConnection().createStatement();
        String query = "INSERT INTO nesfield_student (Roll_no, Name, Email, Address) VALUES " +
            "(7, 'Sagar', 'Sagar@gmail.com', 'Dhangadi'), " +
            "(8, 'Jhalak', 'jhalak@gmail.com', 'makannpur)";
        int result = statement.executeUpdate(query);
        System.out.println(result + " row(s) inserted.");
    }
    catch(SQLException e)
    {
        System.out.println("Error: " + e);
    }
}
}

```

ResultSet

- The JDBC ResultSet is object which represents the result of a SQL, query executed on a database.
- It acts as cursor to navigate through the retrieved data, manipulate data, Fetching specific columns and others.
- ResultSet is essentially a table of data where each row represents a record and each column represents a field in the database.
- The ResultSet has a cursor that points to the current row in the ResultSet and we can able to navigate in ResultSet by using the next(), previous(), first() and last() methods.
- We can retrieve data by using different method like getString(), getInt(), getDouble() and other methods.
- It is the part of JDBC API which is used for interacting with relational databases.
- The ResultSet allows us over the rows of tables returned by the SQL query and extract a specific column from the SQL query result.

Common Operation with ResultSet

- Fetching data from database
- Navigating the ResultSet
- Getting column values
- Closing the result

Types of ResultSet

There are three different characteristics by which ResultSet types are differentiated

1. Scrollability

Determines whether you can move back and forth in the ResultSet

- **TYPE_FORWARD_ONLY:** can only move forward through the rows.
- **TYPE_SCROLL_INSENSITIVE:** can move forward and backward but changes are not reflect ResultSet.
- **TYPE_SCROLL_SENSITIVE:** can move forward and backward but changes are affect the ResultSet.

2. Concurrency

Determines whether you can update the ResultSet.

- **CONCUR_READ_ONLY:** can only read data.
- **CONCUR_UPDATABLE:** allows updates to the ResultSet.

3. Holdability

Determines what happens to the ResultSet when a Transaction is committed

- **HOLD_CURSORS_OVER_COMMIT:** the Resultset remains opens after a commit.
- **CLOSE_CURSORS_AT_COMMIT:** the ResultSet closes after a commit.

Category of Methods in ResultSet

1. Navigating a ResultSet

Method	Description
next()	used for move next row in the ResultSet.
previous()	used for move to previous row in the ResultSet
first()	used for move to first row in the ResultSet
last()	used for move to last row in the ResultSet
absolute(int row)	used to move to specific row
relative(int rows)	used for Moves forward or backward by the specified number of rows
beforeFirst()	used for Positions the cursor before the first row
afterLast()	used for Positions the cursor after the last row

2. Retrieving Data from a ResultSet

Method	Description
getInt(int columnIndex)	used for Retrieves an integer from the specified column
getString(int columnIndex)	used for Retrieves a string from the specified column
getDouble(int columnIndex)	used for Retrieves a double from the specified column
getBoolean(int columnIndex)	used for Retrieves true or false from the specified column
getDate(int columnIndex)	used for Retrieves a java.sql.Date
getObject(int columnIndex)	used for Retrieves any type of object
getArray(int columnIndex)	used for Retrieves a SQL array

3. Updating Data in a ResultSet

Method	Description
<code>updateInt(int columnIndex, int x)</code>	used for Updates an integer value in the specified column
<code>updateString(int columnIndex, String x)</code>	used for Updates a string value
<code>updateBoolean(int columnIndex, boolean x)</code>	used for Updates a boolean value
<code>updateRow()</code>	used for Updates a row
<code>deleteRow()</code>	used for delete a row

Example

Get data from table (explain above) is the simple implementation of ResultSet.

For more example see lab report.

RowSets

- We have been using ResultSet to store query result.
- Use of ResultSet requires programmers to manage the Connection and Statement explicitly.
- JDBC 2.0 introduces a new RowSet interface, which manages Connection and statement for you automatically and greatly simplifies your programming for enterprise web applications.
- To use RowSet:
 - Construct an instance of RowSet.
 - Set the properties and query statement of the RowSet.
 - Invoke `execute()` to execute the query.

Example

```
import java.sql.*;
import javax.sql.rowset.*;

public class RowSet1
{
    public static void main(String args[])
    {
        try
        {
            // Load the MySQL JDBC driver
            String driver = "com.mysql.cj.jdbc.Driver";
            Class.forName(driver);

            // Create a RowSetFactory and JdbcRowSet
            RowSetFactory rsf = RowSetProvider.newFactory();
            JdbcRowSet rs = rsf.createJdbcRowSet();

            // Set database credentials and query
            rs.setUrl("jdbc:mysql://localhost:3306/Nesfield");
            rs.setUsername("root");
            rs.setPassword(""); // Corrected this line
            rs.setCommand("SELECT * FROM nesfield_student");
            rs.execute();

            // Move to the last row and display info
            rs.last();
            System.out.println("Last row information:");
            System.out.println(rs.getInt("roll_no") + ", " + rs.getString("name") + ", " + rs.getString("email") + ", " + rs.getString("address"));

            // Move to the first row and display info
            rs.first();
            System.out.println("First row information:");
            System.out.println(rs.getInt("roll_no") + ", " + rs.getString("name") + ", " + rs.getString("email") + ", " + rs.getString("address"));

            rs.close(); // Always close your RowSet
        }
        catch (Exception e)
        {
            System.out.println("Error: " + e);
        }
    }
}
```

Output

```
Last row information:
4, Eliza, Eliza@gmail.com, Lalitpur
First row information:
1, Nishanta, Nishanta@gmail.com, Bhaktapur
```

Transactions in Java

- There are times when we do not want one statement to take effect unless another one completes.
- For example, when the proprietor of the coffee break updates the amount of coffee sold each week, the proprietor will also want to update the total amount sold to date.
- However, the amount sold per week and the total amount sold should be updated at the same time; otherwise, the data will be inconsistent. The way to be sure that either both actions occur neither action occurs is to use a transaction. A transaction is a set of one or more statements that is executed as a unit, so either all of the statements are executed or none of the statement is executed. The way to allow two or more statements to be grouped into a transaction is to disable the auto – commit mode. This is demonstrated in the following code, where con is an active connection:

Con.setAutoCommit(false)

- After the auto commit mode is disabled, no sql statements are committed until we call the method commit explicitly. All statements executed after the previous call to the method commit are included in the current transaction and committed together as a unit.

Example

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class Transaction1
{
    public static void main(String args[])
    {
        DataTransaction();
    }

    public static Connection getConnection() {
        try {
            String driver = "com.mysql.cj.jdbc.Driver";
            String databaseUrl = "jdbc:mysql://localhost:3306/Nesfield";
            String username = "root";
            String password = "";
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(databaseUrl, username, password);
            System.out.println("Database connected");
            return conn;
        }
        catch (Exception e)
        {
            System.out.println("Some error: " + e);
        }
        return null;
    }
}
```

```

public static void DataTransaction()
{
    Connection conn = getConnection();
    try
    {
        Statement st = getConnection().createStatement();

        conn.setAutoCommit(false);
        String query1 = "update bank set Balance = Balance - 5000 where Acc_no = 105";
        String query2 = "update bank set Balance = Balance + 5000 where Acc_no = 205";
        st.executeUpdate(query1);
        st.executeUpdate(query2);
        conn.commit();

        ResultSet result = st.executeQuery("select * from bank");
        System.out.println("After Transaction Complete");
        while (result.next()) {
            System.out.println(result.getInt("Acc_no"));
            System.out.println(result.getString("Acc_name"));
            System.out.println(result.getFloat("Balance"));
            System.out.println(result.getString("Branch"));
        }
    }
    catch (Exception e)
    {
        System.out.println("Error: " + e);
    }
}

```

output

Database connected

After Transaction Complete

105

Bikram

35000.0

Bhaktapur

107

Subein

5000.0

Kalanki

205

Abishek

35000.0

Gwarko

SQL Injection

- SQL injection is a security vulnerability that occurs when an application or web page uses user input directly in sql queries without proper validation or sanitization. Malicious users can exploit this weakness by inserting specially crafted input that alters the intended SQL query often with malicious intent.

Why SQL injection occurs?

1. User Input Concatenation
User inputs are directly combined with SQL queries without proper validation or escaping
2. Lack of input validation
Failure to validate and sanitize user inputs allows malicious data to manipulate SQL queries.
3. Misunderstanding Trust Level
Assumptions about the trustworthiness of certain inputs can lead to vulnerability.
4. Inadequate Error Handling
Poor error handling exposes application and database details that attackers can exploit.
5. Complex SQL queries
Complex queries with multiple parameters increase the likelihood of injection vulnerabilities.
6. Legacy Code and outdated practices
Older application may lack modern security measures against SQL injection
7. Resource Constraints
Development priorities may favor speed over security, leading to shortcuts
8. Lack of Security Awareness
Insufficient knowledge of security best practices can result in vulnerable code.

Vulnerable code

The code that is recognized as safe can have flow in it too. SO, let us check vulnerable code by starting by examining a vulnerable piece of code.

```
public static void getData()
{
    try
    {
        Statement statement = getConnection().createStatement();
        ResultSet result = statement.executeQuery("SELECT * FROM nesfield_student where roll_no = 3 or '1=1'");
        while (result.next()) {
            System.out.println(result.getString("Roll_no"));
            System.out.println(result.getString("Name"));
            System.out.println(result.getString("Email"));
            System.out.println(result.getString("Address"));
        }
    }
    catch (Exception e) {
        System.out.println("Error: " + e);
    }
}
```

In this code the roll_no is directly put into the SQL query string, creating a glaring vulnerability.

Preventing SQL Injection

To prevent SQL injection, always use prepared statement with parameterized queries as below example.

```
import java.sql.*;

public class sql_injection
{
    public static void main(String args[])
    {
        SelectData();
    }

    public static Connection getConnection()
    {
        try
        {
            String driver = "com.mysql.cj.jdbc.Driver";
            String databaseUrl = "jdbc:mysql://localhost:3306/nesfield";
            String username = "root";
            String password = "";
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(databaseUrl, username, password);
            System.out.println("Database connected");
            return conn;
        }
        catch (Exception e)
        {
            System.out.println("Some error: " + e);
        }
        return null;
    }
}
```

```

public static void SelectData()
{
    Connection conn = null;
    PreparedStatement ps = null;
    ResultSet result = null;
    try
    {
        conn = getConnection();
        if (conn != null)
        {
            String sql = "SELECT * FROM nesfield_student WHERE Roll_no = ?";
            ps = conn.prepareStatement(sql);
            ps.setInt(1, 3);
            result = ps.executeQuery();

            while (result.next()) {
                System.out.println(result.getString("Roll_no"));
                System.out.println(result.getString("Name"));
                System.out.println(result.getString("Email"));
                System.out.println(result.getString("Address"));
            }
        }
    }
    catch (Exception e)
    {
        System.out.println("Error: " + e);
    }
}

```

In this secure version, we use a prepared statement with a ‘?’ as a placeholder for the ‘roll_no’ value. The ‘setString’ method safely binds the parameter to the query ensuring that any malicious input is treated as data not code.

SQL Escapes

- Escapes sequences are used within an SQL statement to tell the driver that the escaped part of the SQL string should be handled differently.
- When the JDBC driver processes the escaped part of an SQL string, it translates that part of the string into SQL code that SQL Server understands.
- There are five types of escape sequences that the JDBC API requires, and all are supported by the JDBC driver:
 - LIKE wildcard literals
 - Function Handling
 - Date and Time literals

- Stored Procedure calls
- Outer joins

Like wildcard Literals

The JDBC driver supports the {escape 'escape character'} syntax for using LIKE clause wildcards as literals. For example, the following code will return value for col3, where the value of col2 literally begins with an underscore.

```
ResultSet rst = stmt.executeQuery("SELECT col3 FROM test1 WHERE col2  
LIKE '\\_%' {escape '\\'}");
```

Function Handling

The JDBC driver supports function escape sequences in SQL statements with the following syntax:

```
{fn functionName}
```

Where functionName is a function supported by the JDBC driver. For example

```
SELECT {fn UCASE(Name)} FROM Employee
```

Date and Time literals

The escape syntax for date, time and timestamp literals is the following

```
{literal-type 'value'}
```

Where literal – type is one of the following:

Literal Type	Description	Value Format
d	Date	yyyy-mm-dd
t	Time	hh:mm:ss [1]
ts	TimeStamp	yyyy-mm-dd hh:mm:ss[f...]

For example

```
UPDATE Orders SET OpenDate={d '2005-01-31'}  
WHERE OrderID=1025
```

Stored Procedure calls

The JDBC driver supports the `{? = call proc_name(?,...)}` and `{call proc_name(?,...)}` escape syntax for stored procedure calls, depending on whether you need to process a return parameter.

A procedure is an executable object stored in the database. Generally, it is one or more SQL statements that have been precompiled. The escape sequence syntax for calling a stored procedure as follows:

```
{[?]=call procedure-name([parameter][,parameter]...)]}
```

Where procedure – name specifies the name of a stored procedure and parameter specifies a stored procedure parameter.

Outer Joins

The JDBC driver supports the SQL left, right, and full outer join syntax. The escape sequence for outer joins is the following:

```
{oj outer-join}
```

Where outer join is

```
table-reference {LEFT | RIGHT | FULL} OUTER JOIN  
{table-reference | outer-join} ON search-condition
```

Where table –reference is a table name and search – condition is the join condition you want to use for the tables

```
SELECT Customers.CustID, Customers.Name, Orders.OrderID, Orders.Status  
FROM {oj Customers LEFT OUTER JOIN  
      Orders ON Customers.CustID=Orders.CustID}  
WHERE Orders.Status='OPEN'
```